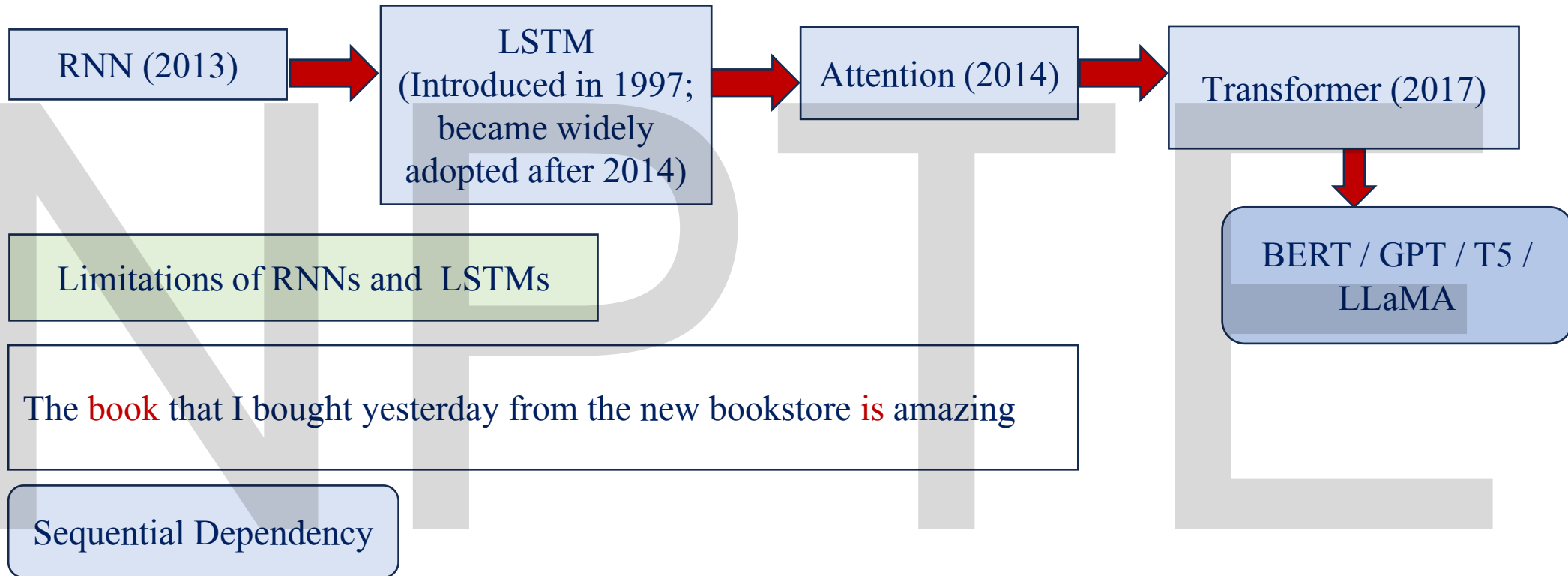


Evolution From LSTMs to Transformers

Contents

- ✓ Evolution of Sequence Models
- ✓ Limitations of RNNs and LSTMs
- ✓ Attention Mechanism
- ✓ Encoder–Decoder Attention
- ✓ Self-Attention
- ✓ Multi-Head Attention
- ✓ Masked Self-Attention
- ✓ Introduction to Transformers
- ✓ RNN/LSTM vs Transformer

The Evolution of Sequence Modeling in Deep Learning



- Recurrent models generate a sequence of hidden states h_t .

$$h_t = \tanh(vh_{t-1} + Ux_t + b_h)$$

Limitations of RNNs and LSTMs

Sequential Dependency

Memory Constraints

- For long sequences, the computer must hold a chain of these states to ensure information from the beginning of the sentence eventually reaches the end.

Problem with Long Range Dependency

- ✓ Information between distant words must travel through $O(n)$ sequential steps.
- ✓ If there are 1000 words in a long sequence, it will involve 1000 operations

"The quick brown fox.."



To understand the word **brown**, the model must first process **quick**, which depends on **The**.

Motivation for Attention

Attention Idea

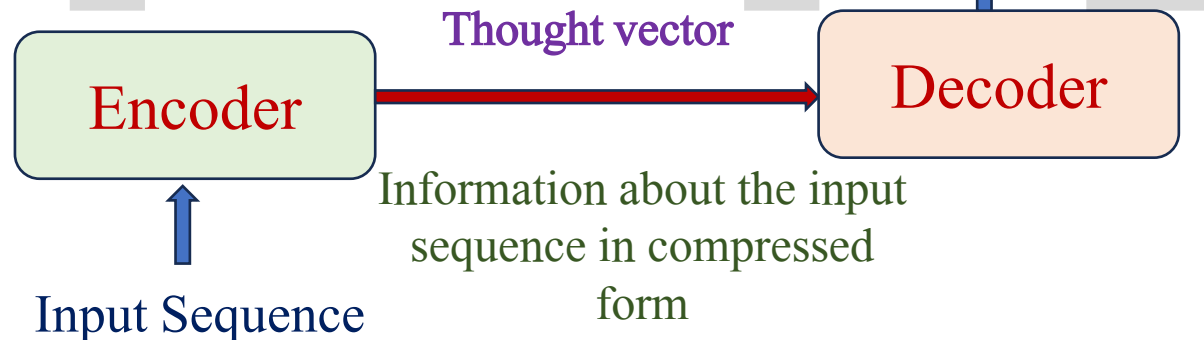
When predicting a word, do we really need every previous hidden state equally?

Answer: No.

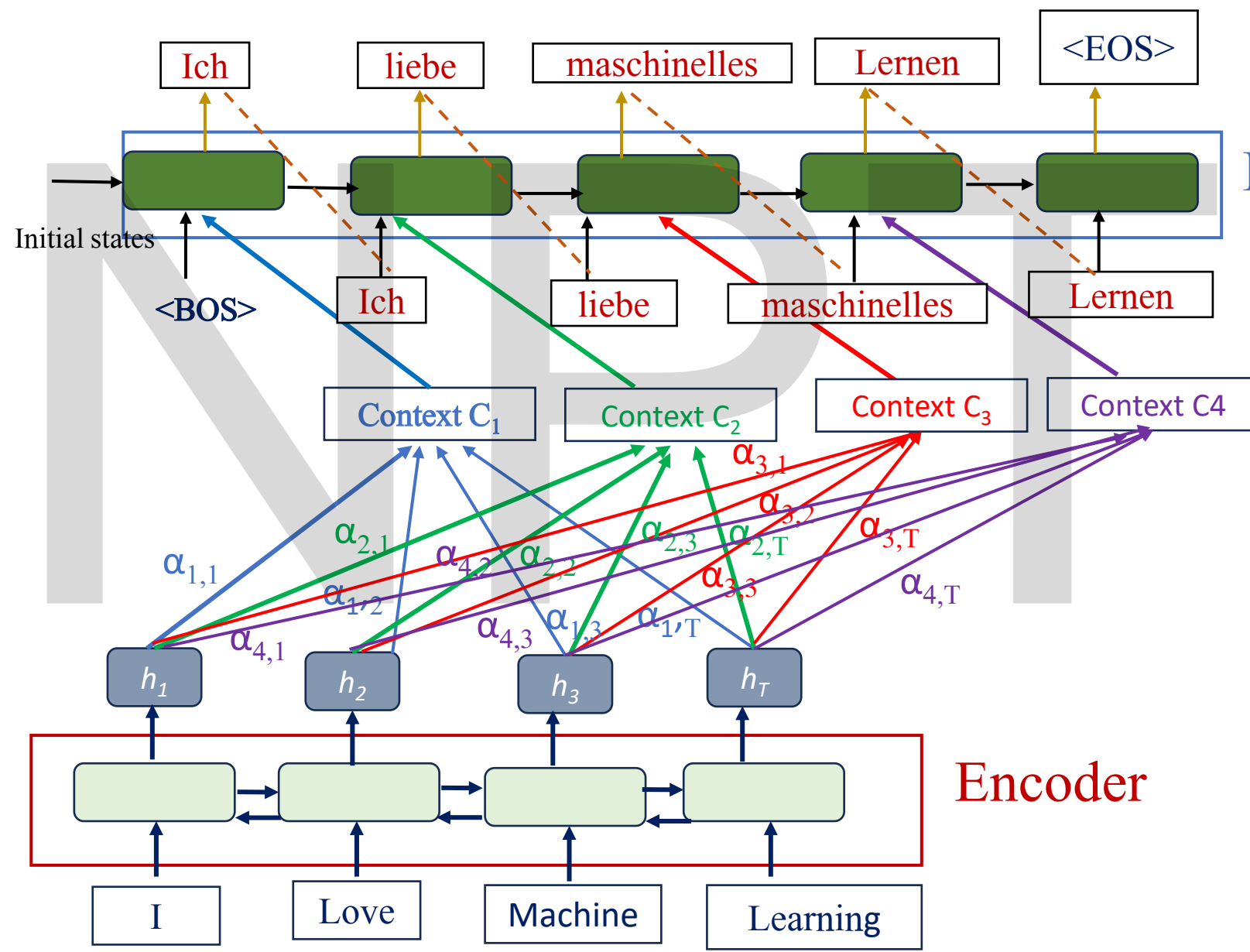
The model should assign greater importance to the most relevant words in the sequence.

The **animal** didn't cross the street because **it** was too tired.

Attention with ENCODER-DECODER Architecture



Encoder Decoder with Attention



α denotes the amount of attention to be paid to each hidden state

$$\sum_{j=1}^T \alpha_{t,j} = 1$$

$\alpha_{t,j}$ = attention weight assigned to the j^{th} input word while generating the t^{th} output word.

Decoder

$$C_1 = \sum_{j=1}^T \alpha_{1,j} h_j$$

$$C_2 = \sum_{j=1}^T \alpha_{2,j} h_j$$

$$C_3 = \sum_{j=1}^T \alpha_{3,j} h_j$$

$$C_4 = \sum_{j=1}^T \alpha_{4,j} h_j$$

$$C_t = \sum_{j=1}^T \alpha_{t,j} h_j$$

Encoder

From Attention to Self-Attention

In Encoder –Decoder attention, the attention mechanism is between the input words(from the Encoder) and the outputs (from the decoder)

Core idea of Self Attention

- Every word learns from every other word.
- Each token is projected into three vectors:

Query(Q)

Key (K)

Value (V)

Example:

I love Generative AI

Tokens

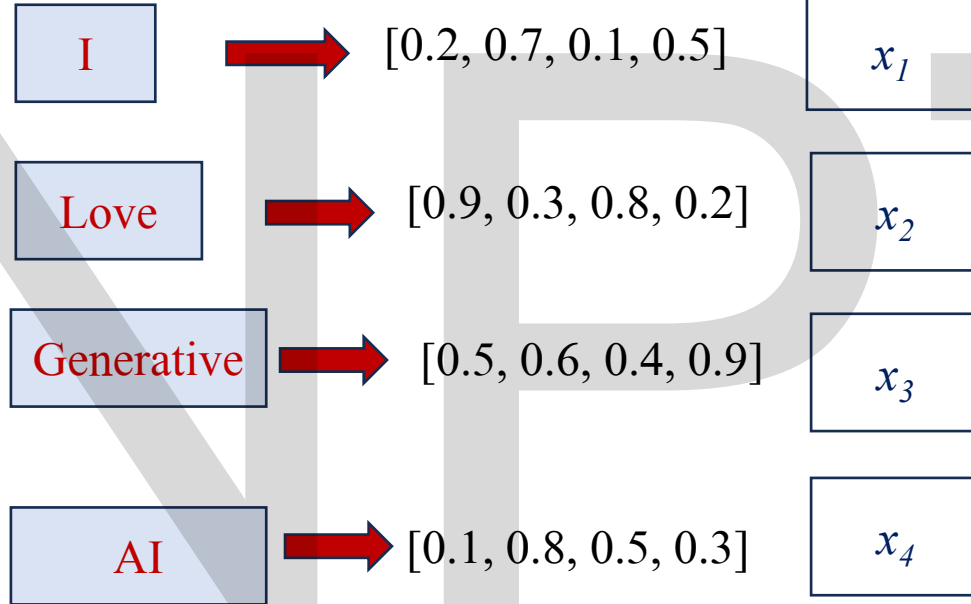
'I', 'Love', 'Generative', 'AI'

Each of these tokens will have embedding vectors.

Example

Token

Embedding Vectors



W_Q : weight matrices associated with query vector

W_k : weight matrices associated with key vector

W_v : weight matrices associated with value vector.

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Each word will have a Query vector (Q), Key vector(K) and Value vector(V)

$$\begin{aligned} Q &= XW_Q \\ K &= XW_K \\ V &= XW_V \end{aligned}$$

Multi-Head Attention

Motivation

"The scientist published the paper because she completed the experiment successfully."

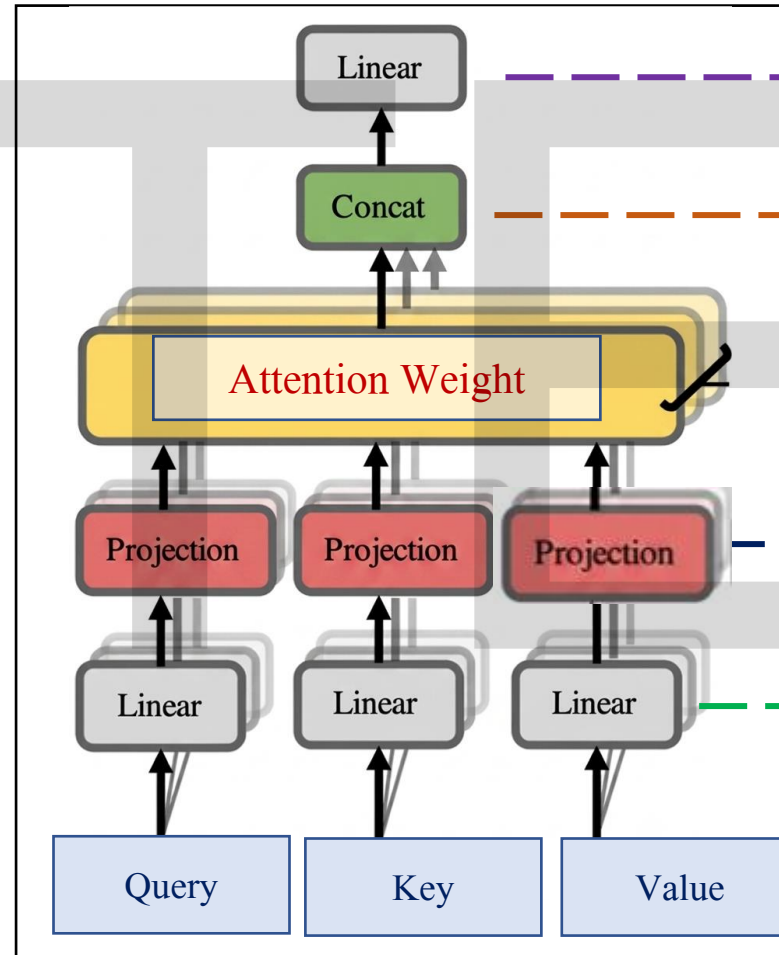
paper ↔ published
experiment ↔ completed
scientist ↔ completed

A single attention head can only learn one dominant relationship at a time.

Core idea of Multi-Head attention

Different attention heads learn different linguistic relationships simultaneously.

- ✓ Query (What am I looking for?)
- ✓ Key (What information do I contain?)
- ✓ Value (What information should I pass?)



combines all these pieces of information into one final representation.

stack all outputs together to have one large feature vector.

projections create the Query, Key and Value representations for **one** attention head.

Linear layers use different learnable weight matrices to create different views of the same sentence.

Masked Self Attention

English to French translation

I am a student

"Je suis un étudiant"

Je suis un

étudiant is getting predicted

Encoder

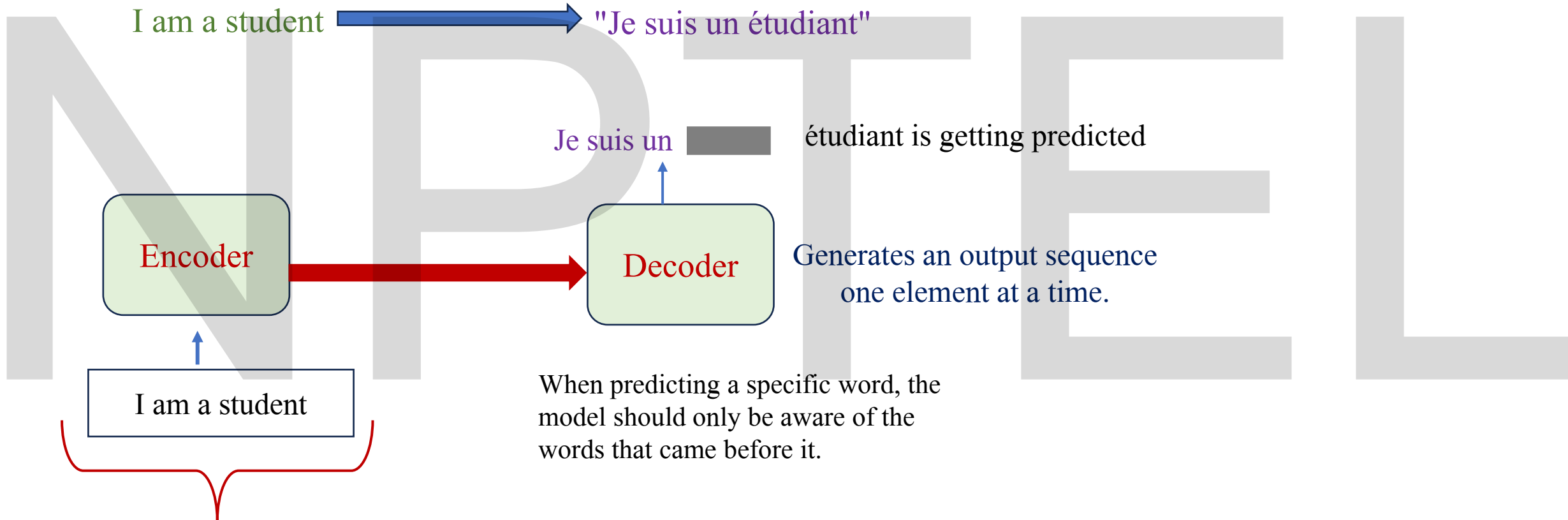
Decoder

Generates an output sequence
one element at a time.

I am a student

When predicting a specific word, the
model should only be aware of the
words that came before it.

Entire sentence is given at once



Attention matrix without masking

Query/Key	I	Love	Deep	Learning
I	2	4	1	3
Love	1	5	2	6
Deep	3	2	5	4
Learning	2	3	1	6

Attention matrix with masking

Query/Key	I	Love	Deep	Learning
I	2	$-\infty$	$-\infty$	$-\infty$
Love	1	5	$-\infty$	$-\infty$
Deep	3	2	5	$-\infty$
Learning	2	3	1	6

Every token can attend to

- previous tokens
- itself
- **Never future tokens.**

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Softmax function

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Breakthrough in 2017- Introduction to Transformers

The Landmark Paper

"Attention Is All You Need" by Vaswani et al., Google Brain (2017)

Introduced:
The Transformer architecture which works based on

Attention Mechanism

Self Attention

Multi-head
Attention

Encoder Decoder
Attention

Parallel Processing

RNNS/LSTMs Vs Transformers

Core Idea:

Instead of reading one word after another, it looks at **all words simultaneously**.

RNN/LSTM

Reads sequentially

Hidden states carry memory

Difficult to parallelize

Slow training for long sequences

Long-distance information may weaken

Transformer

Reads all tokens together

Attention connects tokens directly

Highly parallelizable

Faster Training

Long-distance dependencies are handled effectively

NPTEL

Summary

NLP T E L

Next Session:

Transformer Architecture